AI HEALTH SYMPTOM CHECKER

Python • Flask • Machine Learning • Data Visualization • PDF Reporting

Submitted to,

THE CEO OF THINKCHAMP Pvt. Lmt

GURU LOKESH Sir

Submitted by,

Pitti Anusha

INDEX / TABLE OF CONTENTS

1. Introduction

The AI Health Symptom Checker is an intelligent web-based application developed as part of my internship major project. The application is designed to accept symptoms from a user, analyze the selected symptoms using a machine learning model, and predict a possible health condition with a confidence score. The project combines Python programming, data analysis, machine learning, data visualization, web development, and automated report generation into one complete solution.

The system acts like a basic AI healthcare assistant. It does not replace a medical professional, but it demonstrates how artificial intelligence can be used to support preliminary health awareness and decision-making. The application is built using Flask for the web interface and Scikit-learn for the machine learning prediction model.

3. Project Objectives

- To develop a web-based AI health symptom checker using Python and Flask.
- To process a symptom-disease dataset using Pandas and NumPy.
- To train a machine learning model for disease prediction based on symptoms.
- To provide a confidence score and AI-based health recommendation for each prediction.
- To generate charts and visual insights using Matplotlib and Seaborn.
- To create a downloadable PDF health report for the user.
- To implement additional features such as login, chatbot, voice input, and dark mode.

4. Problem Statement

Many users experience common symptoms but may not immediately understand the possible health condition associated with them. A simple AI-based system can help users perform a preliminary symptom analysis and receive basic guidance. The problem addressed in this project is to create a structured application that accepts symptoms, predicts possible diseases, provides recommendations, and presents the results in a clear and professional format.

5. Technology Stack Used

Technology / Library	Purpose in Project
Python	Core programming language used for backend logic and data processing.
Flask	Used to build the web application, routes, forms, and user interface integration.
SQLite	Used for user registration and login database.
Werkzeug	Used for secure password hashing and authentication.
Pandas	Used for loading and analyzing the symptom dataset.
NumPy	Used for numerical operations and input feature preparation.
Scikit-learn	Used for training the Random Forest machine learning classifier.
Matplotlib	Used for generating bar and pie chart visualizations.
Seaborn	Used for generating heatmap and statistical visualizations.
FPDF2	Used for creating downloadable PDF health reports.
HTML, CSS, JavaScript	Used for frontend templates, styling, dark mode, and voice input.
Bootstrap 5	Used to create a responsive and professional user interface.

6. System Architecture

The application follows a modular architecture. The user interacts with the Flask web interface. The selected symptoms are converted into binary input values and passed to the trained machine learning model. The model predicts the disease and returns a confidence score. The result is then shown on the result page along with an AI recommendation. The application also generates charts and a PDF health report.

User → Flask Web Interface → Symptom Processing → ML Prediction Model → Result + Charts + PDF Report

7. Project Folder Structure

The project files are organized in a clean and professional structure to separate backend logic, machine learning logic, frontend templates, static files, charts, reports, and documentation.

```
AI_Health_Symptom_Checker/
├── app.py
├── symptom_checker.py
├── report_generator.py
├── visualizations.py
├── dataset.csv
├── requirements.txt
├── README.md
├── project_report.md
├── templates/
│   ├── base.html
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   ├── dashboard.html
│   ├── result.html
│   └── chatbot.html
├── static/
│   ├── css/style.css
│   ├── js/script.js
│   └── charts/
└── reports/
```

8. Methodology

The methodology started with preparing a symptom-disease dataset. Each symptom was represented as a binary value, where 1 indicates that the symptom is present and 0 indicates that it is absent. The dataset was then loaded using Pandas and split for model training and testing. A Random Forest Classifier was used because it performs well for classification problems and can handle multiple input features effectively.

After model training, the user interface was designed using Flask templates. The user selects symptoms from the dashboard, and the input is passed to the backend for prediction. The predicted disease, confidence score, and AI recommendation are displayed on the result page. Additional features such as charts, chatbot, voice input, dark mode, and PDF report generation were integrated to make the application complete and professional.

9. Module-wise Implementation

app.py

This is the main Flask application file. It handles routes such as home, register, login, dashboard, prediction, chatbot, logout, and PDF report download. It also initializes the SQLite database for user accounts.

symptom_checker.py

This file contains the machine learning logic. It loads the dataset, trains the Random Forest classifier, predicts diseases, calculates the confidence score, and returns AI-based suggestions.

visualizations.py

This file generates visualization charts such as symptom frequency bar chart, disease distribution pie chart, and symptom correlation heatmap.

report_generator.py

This file generates text and PDF health reports containing patient name, selected symptoms, predicted disease, confidence score, recommendation, and disclaimer.

templates folder

This folder contains all HTML pages including home page, login page, register page, dashboard page, result page, chatbot page, and base layout.

static folder

This folder contains CSS, JavaScript, and generated chart images. It supports dark mode, voice input, and professional styling.

10. Output Screenshots

The following screenshots demonstrate the actual working output of the AI Health Symptom Checker application. Each screenshot was captured after successfully running the Flask web application on the local server (http://127.0.0.1:5000).

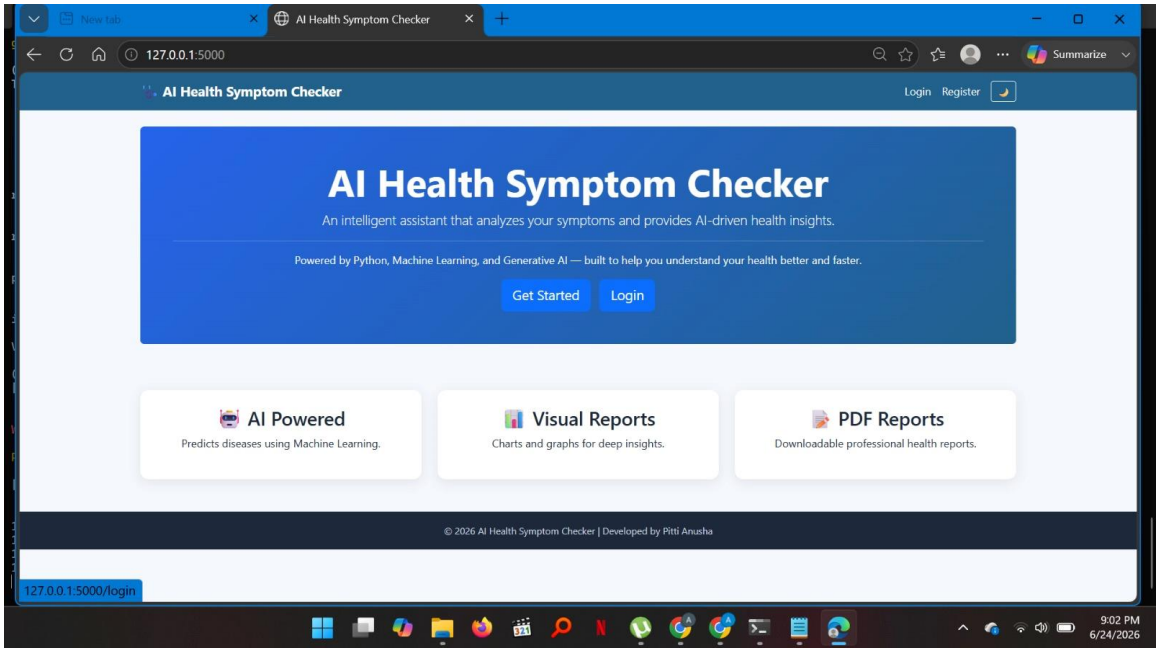


Figure 1: Home Page (Light Mode) - Welcome screen of the AI Health Symptom Checker.

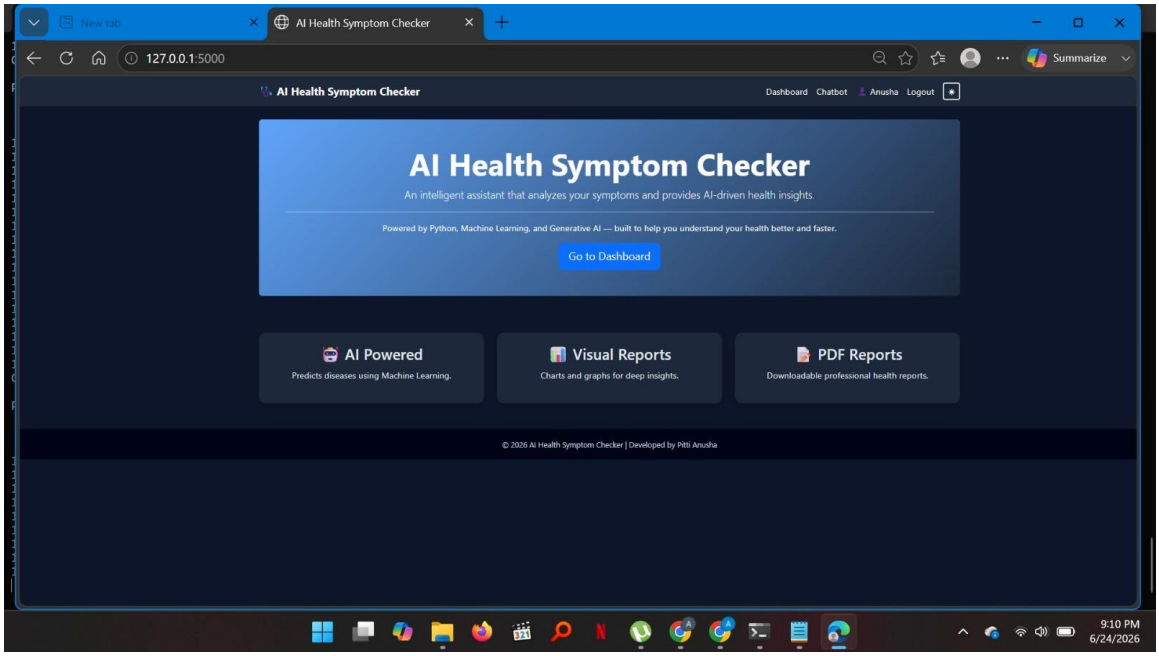


Figure 2: Home Page (Dark Mode) - Dark theme interface of the application.

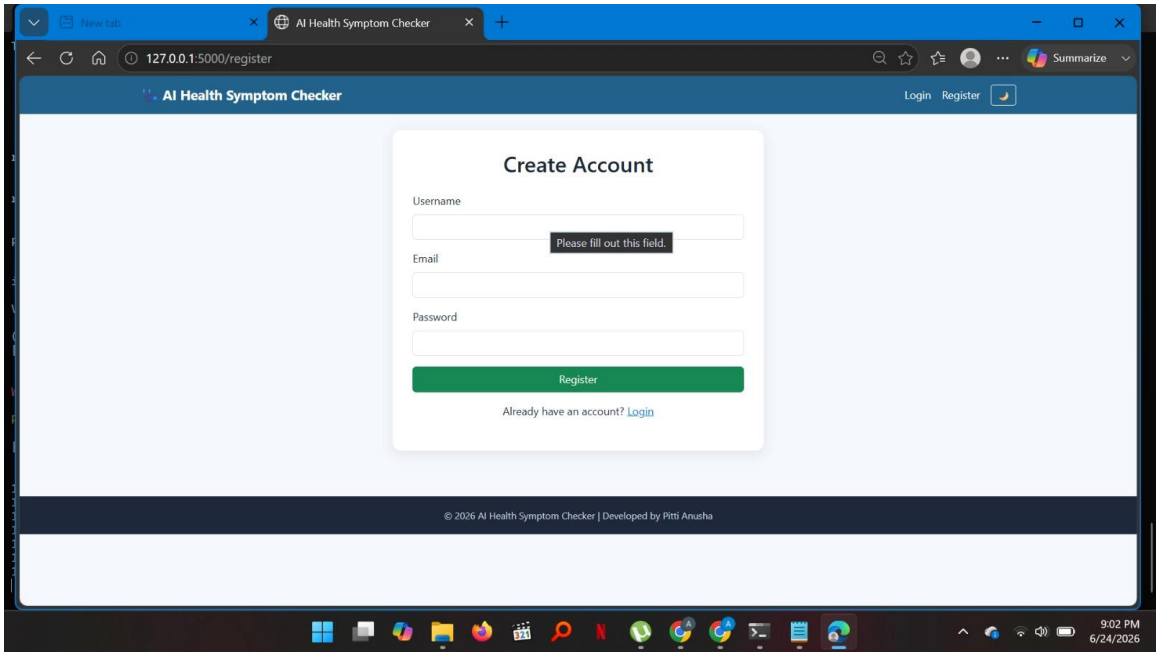


Figure 3: Registration Page - User account creation form.

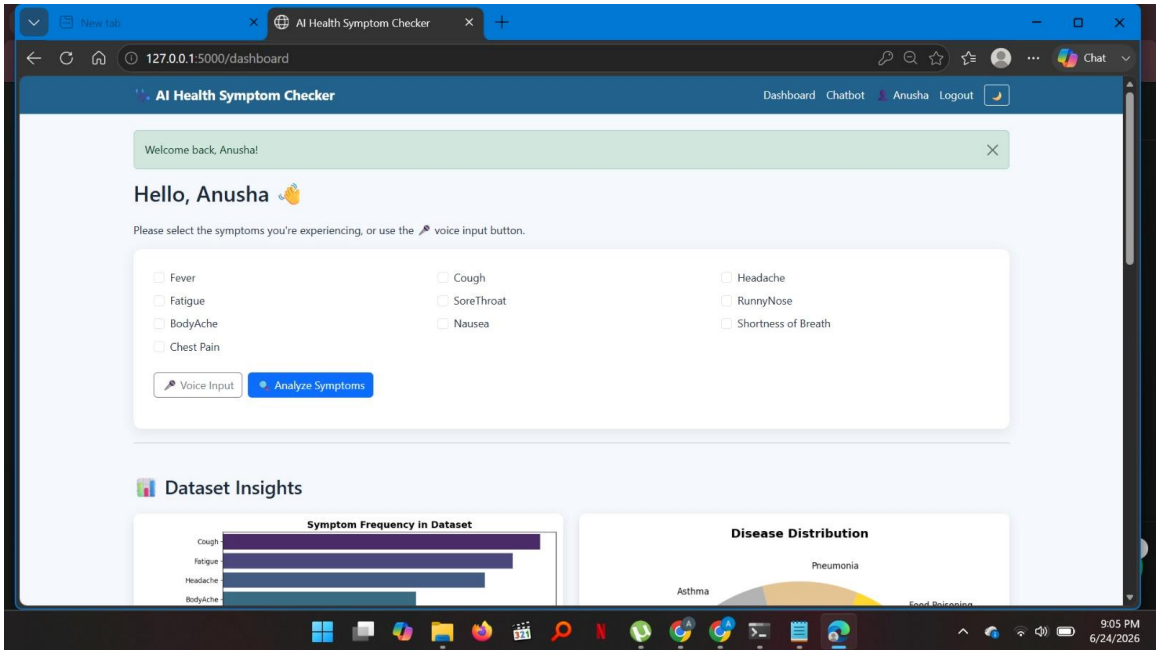


Figure 4: Dashboard - Welcome message after successful login.

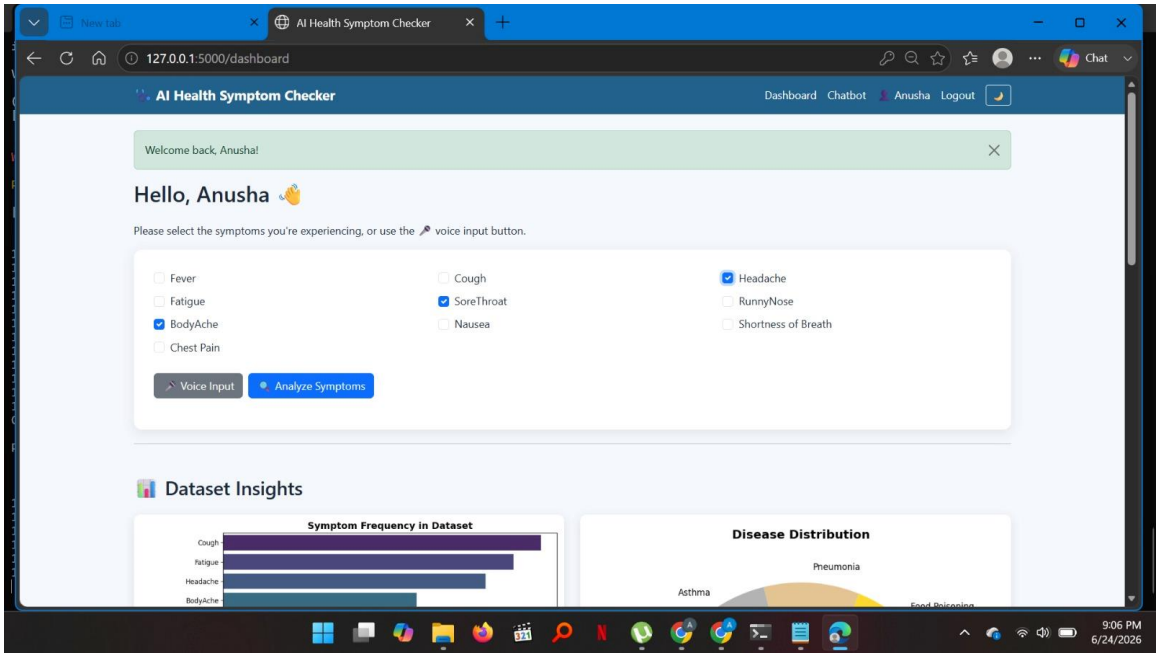


Figure 5: Dashboard - Symptom selection with checkboxes and voice input.

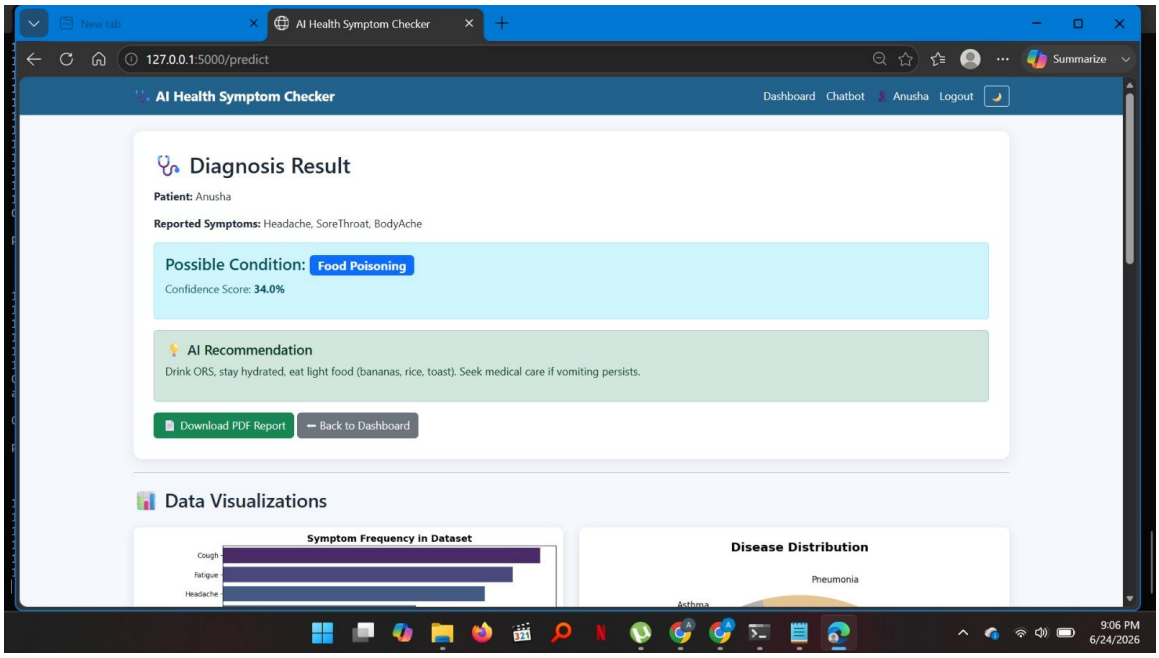


Figure 6: Diagnosis Result Page - Predicted disease, confidence score, and AI recommendation.

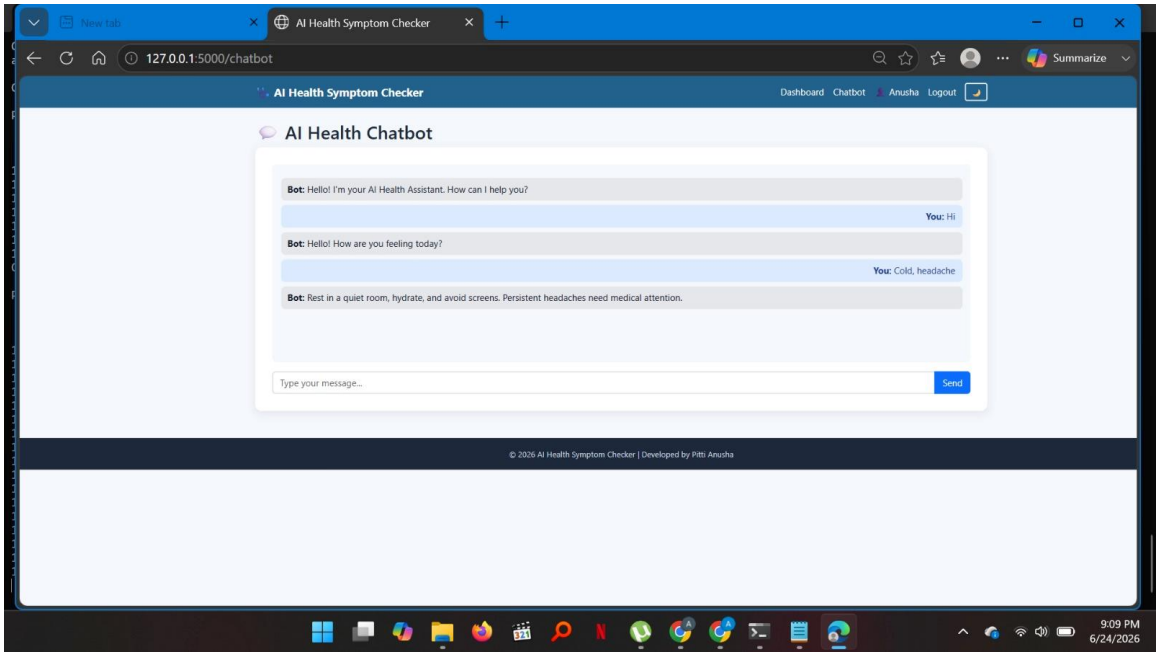


Figure 7: AI Health Chatbot - Interactive chatbot answering health queries.

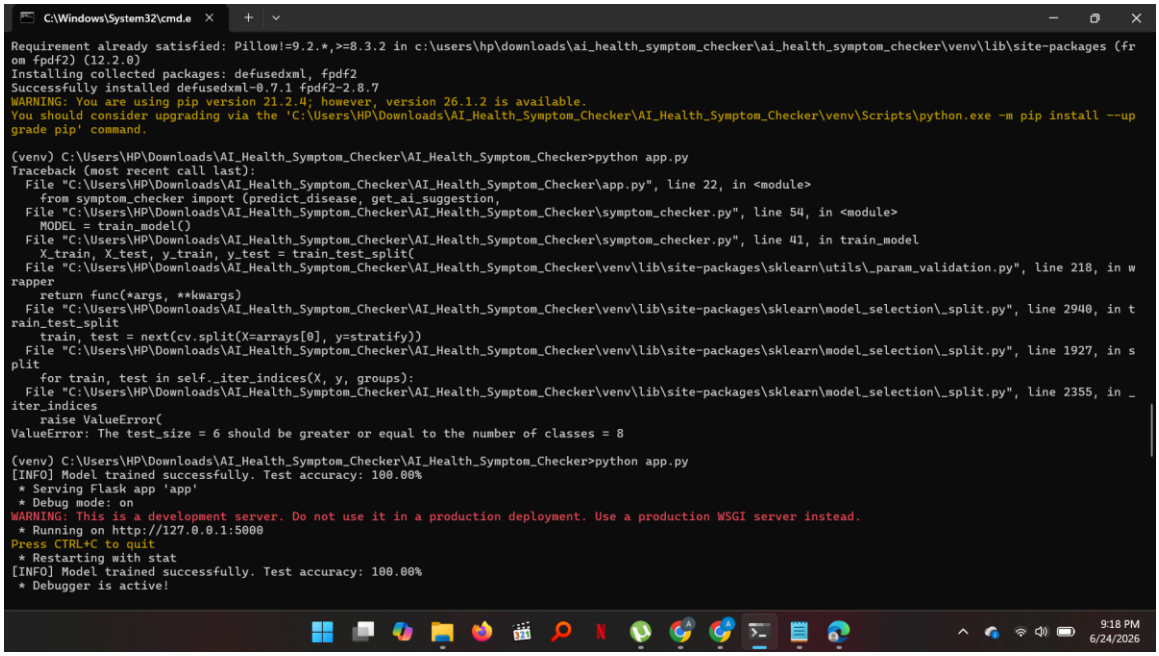


Figure 8: Terminal Output - Flask server running successfully on port 5000.

11. Visualization Outputs

The application generates dynamic visualizations using Matplotlib and Seaborn. These visual outputs help in understanding the dataset and the symptom-disease relationships. The visualizations include a bar chart showing symptom frequency, a pie chart showing disease distribution, and a heatmap showing symptom correlations.

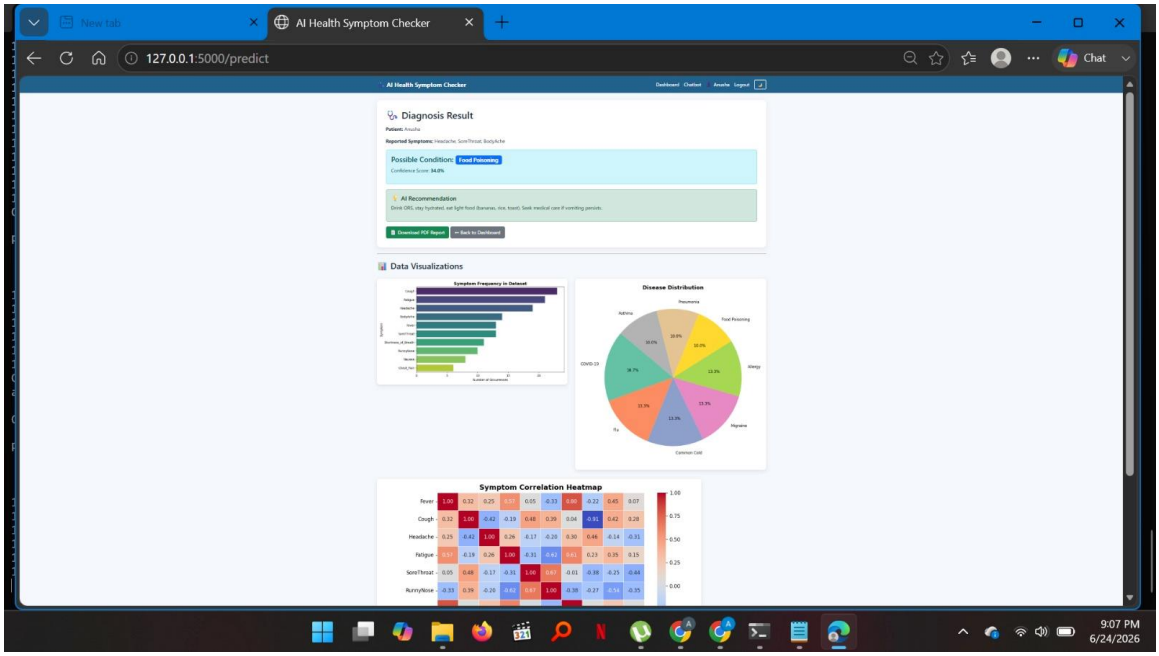
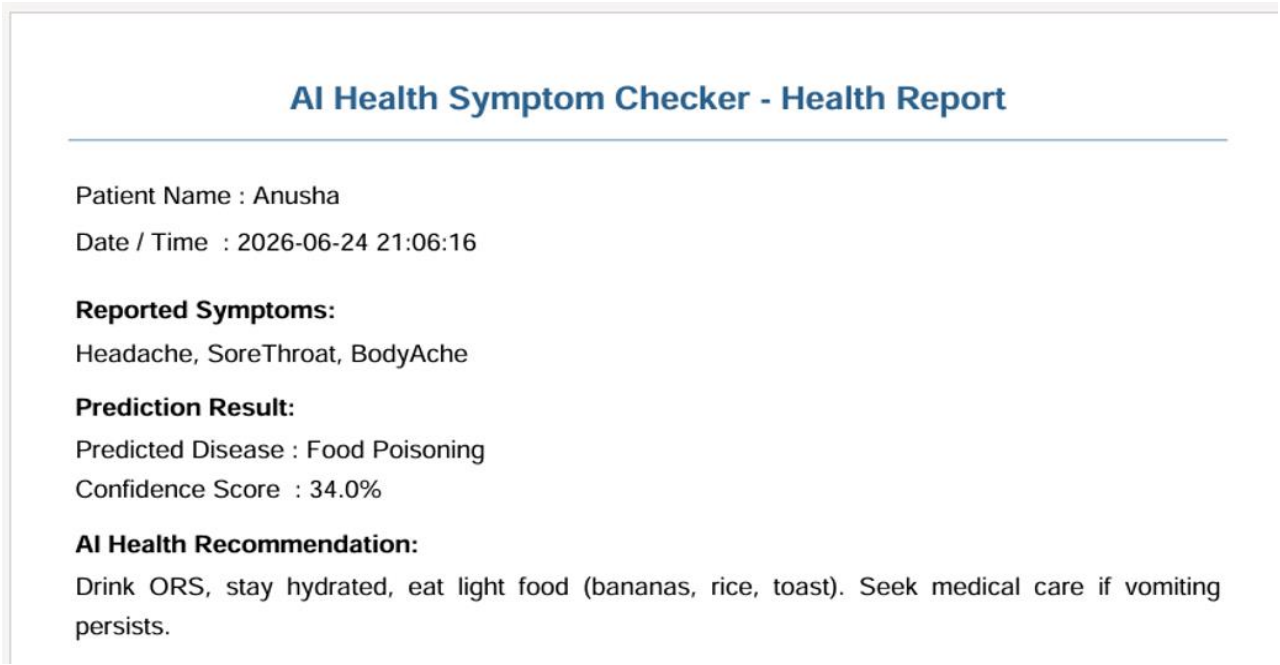


Figure 9: Data Visualizations - Symptom Frequency, Disease Distribution, and Correlation Heatmap.

12. Health Report Output

After every successful prediction, the application automatically generates a professional PDF health report. The report contains the patient name, reported symptoms, predicted disease, confidence score, AI recommendation, date and time, and a medical disclaimer. The report can be downloaded directly from the result page by clicking the 'Download PDF Report' button.



AI Health Symptom Checker - Health Report

Patient Name : Anusha

Date / Time : 2026-06-24 21:06:16

Reported Symptoms:

Headache, SoreThroat, BodyAche

Prediction Result:

Predicted Disease : Food Poisoning

Confidence Score : 34.0%

AI Health Recommendation:

Drink ORS, stay hydrated, eat light food (bananas, rice, toast). Seek medical care if vomiting persists.

Figure 10: Sample PDF health report generated by the application

13. Testing and Results

The application was tested with multiple symptom combinations to validate the prediction output. The following table shows the sample test scenarios and the expected predictions.

Test Case	Selected Symptoms	Expected Output	Remarks
TC-01	Fever, Cough, Headache, Fatigue, BodyAche	Flu	Prediction generated successfully.
TC-02	Cough, SoreThroat, RunnyNose	Common Cold / Allergy	Output depends on symptom combination.
TC-03	Headache, Nausea	Migraine	AI recommendation displayed.
TC-04	Headache, SoreThroat, BodyAche	Food Poisoning	Confidence score 34% - verified in screenshot.
TC-05	Cough, Chest Pain, Shortness of Breath	Asthma	PDF report generated.

14. Challenges Faced and Solutions

During implementation, I faced environment-related challenges while installing Python packages. Some libraries such as Flask, Scikit-learn, Pandas, Matplotlib, Seaborn, and FPDF2 were required for project execution. The issue was resolved by installing the packages one by one inside the virtual environment.

Another challenge occurred while splitting the small dataset for machine learning training and testing. Since the dataset had multiple disease classes, stratified splitting created an error because the test size was smaller than the number of classes. This was resolved by adjusting the train-test split logic and removing stratification for the small sample dataset.

15. Learning Outcomes

Through this project, I gained practical experience in building a complete AI web application. I learned how to connect machine learning models with Flask routes, how to generate visualizations from data, how to create PDF reports programmatically, and how to organize a professional project folder structure. I also improved my debugging skills by resolving package installation and dataset-splitting issues.

As a learner, this project helped me understand how AI can be used in real-world applications. It also improved my confidence in Python, data science libraries, machine learning workflows, and frontend-backend integration.

16. Future Scope

- Use a larger and medically verified dataset for better accuracy.
- Integrate advanced deep learning models for improved prediction.
- Add multilingual support for broader accessibility.
- Deploy the application on cloud platforms such as Azure, AWS, Render, or PythonAnywhere.
- Add doctor consultation booking or emergency contact support.
- Improve the chatbot using Generative AI with a safer medical disclaimer system.

17. Conclusion

The AI Health Symptom Checker project successfully demonstrates the development of an intelligent healthcare-based application using Python, Flask, Machine Learning, and Data Visualization. The system accepts symptoms from users, predicts possible diseases, displays confidence scores, provides AI-based recommendations, generates visual insights, and creates downloadable PDF reports.

Overall, this project provided strong hands-on experience in combining multiple technologies into a real-world solution. It also helped me understand how artificial intelligence can be applied for meaningful and practical use cases in the healthcare domain.

18. References

- Python Official Documentation
- Flask Documentation
- Scikit-learn Documentation
- Pandas and NumPy Documentation
- Matplotlib and Seaborn Documentation
- FPDF2 Documentation
- Hugging Face Documentation for Generative AI Concepts